DEVSECOPS LAB BOOK · PHASE 1: FOUNDATION

Chapter 03

LLDAP + Authentik Modern Identity Provider

From one directory of users to single sign-on everywhere: LDAP, POSIX accounts, SSH key auth, sudo groups, and TLS — built step by step.

LLDAP · SSSD · PAM · OIDC · LDAPS

3.1 — THE PROBLEM

Why Authentik Exists

Authentication
"Who are you?"

Authorization
"What are you allowed to do?"



The Key Ring Analogy

Before key cards, you carried a ring of 50 keys — one per room, changed on different schedules, replaced quarterly. Key cards (SSO/OIDC) mean one card opens everything you're allowed to access.



2005: The Password Sprawl

- Email, bank, LinkedIn
- Project tool, file sharing
- GitLab, Slack, VPN

8 passwords. 8 policies. 8 ways to get breached.

3.1 — HISTORY

The Evolution of Identity Protocols

1993

LDAP

Centralized directory. One username/password everywhere. Authentication only — no authorization or consent.

2012

OAuth2

Delegated access via tokens, not passwords. "Sign in with Google" → app gets a scoped token, not your credentials.

2014

OIDC

OAuth2 + a standard identity layer. Adds an id_token (JWT) and scopes: openid, profile, email, groups.

Today

Authentik

OIDC provider + policy engine + consent manager, backed by LLDAP for user data.

3.1 — ARCHITECTURE

How a Login Flows Through the Stack



User



GitLab



Authentik



LLDAP

1. User clicks "Sign in with OIDC" on GitLab
2. GitLab redirects the browser to Authentik's /authorize endpoint
3. User logs in directly at Authentik
4. Authentik queries LLDAP: "Is this user valid?"
5. LLDAP replies with the confirmed identity (e.g. john@example.com)
6. Authentik redirects back to GitLab with an id_token
7. GitLab now knows exactly who signed in

3.2 — LLDAP

The Lightweight LDAP Server

LLDAP is a modern reimplementation of LDAP in Rust — lightweight, fast, and well suited to a lab. It stores users, groups, and basic attributes while still speaking the LDAP protocol.

Web UI	Manage users & groups visually — no hand-edited LDIF files
Easy backup	Simple data directory, easy to snapshot and restore
Lightweight	Uses far less memory than full OpenLDAP
Still LDAP	Compatible with anything that expects standard LDAP

vs. OpenLDAP: powerful, but complex config files that look like line noise. LLDAP trades some flexibility for usability.

PART 1 — SEC-IDM-01

Deploying LLDAP with Docker Compose

1 · Install Docker + Compose

```
sudo apt install -y docker.io \
  docker-compose
sudo systemctl enable --now docker
```

2 · Create Directory

```
mkdir -p /opt/lldap
cd /opt/lldap
```

3 · docker-compose.yml (key settings)

```
image: ghcr.io/lldap/lldap:stable
ports: 3890 (LDAP)  ·  17170 (Web UI)
environment:
  LLDAP_LDAP_BASE_DN=dc=devsecops,dc=com
  LLDAP_LDAP_USER_PASS=SuperSecureLDAPAdminPass123!
  LLDAP_JWT_SECRET=${LLDAP_JWT_SECRET}
```

Start it: docker-compose up -d → Web UI: http://10.0.2.11:17170

PART 1 — SEC-IDM-01

First Login & LDAP Connection Details

Web UI Login

```
Username:  admin
Password:  SuperSecureLDAPAdminPass123!
```

Then, inside the UI:

- Create groups (e.g. devops)
- Create users (e.g. alex)
- Assign users to their group

Setting	Value
LDAP URI	ldap://10.0.2.11:3890
Base DN	dc=devsecops,dc=com
Bind DN	uid=admin,ou=people,dc=devsecops,dc=com

Note: the Chapter 02 DNS lab uses dc=devsecops,dc=com consistently — double-check the base DN matches across LLDAP, SSSD, and any doc examples referencing dc=example,dc=com.

PART 2 — SEC-VCS-01

LDAP Client: Installing SSSD

```
sudo apt install -y sssd \
  libnss-sss libpam-sss ldap-utils
```

Attribute Mapping (LDAP → POSIX)

```
ldap_user_name = uid
ldap_user_uid_number = uidNumber
ldap_user_gid_number = gidNumber
ldap_user_home_directory = homeDirectory
ldap_user_shell = loginShell
ldap_group_object_class = groupOfNames
```

Key sssd.conf Settings

`id_provider / auth_provider`

ldap — SSSD talks to LLDAP for both identity + auth

`ldap_search_base`

dc=devsecops,dc=com

`ldap_user_search_base`

ou=people,dc=devsecops,dc=com

`ldap_group_search_base`

ou=groups,dc=devsecops,dc=com

`chmod 600 /etc/sssd/sssd.conf` → `sudo systemctl enable --now sssd`

PART 2 — SEC-VCS-01

Wiring NSS & PAM to SSSD

/etc/nsswitch.conf

```
passwd:  files systemd sss
group:   files systemd sss
shadow:  files sss
```

Tells the OS to check SSSD (and thus LDAP) when resolving users, groups, and passwords.

PAM — pam-auth-update

✅ SSS authentication

✅ Create home directory on login

Auto-creates a home dir the first time a synced user logs in — no manual mkdir needed.

Result: any LDAP user in ou=people,dc=devsecops,dc=com becomes a real Linux account — no local /etc/passwd entry required.

```
Verify: getent passwd alex
```

PART 3

Enabling SSH Login via LDAP

1	**sshd_config**	`UsePAM yes` `PasswordAuthentication yes`
2	**/etc/pam.d/sshd**	`auth sufficient pam_sss.so` `account sufficient pam_sss.so`
3	**sshd_config (harden)**	`ChallengeResponseAuthentication no`
4	**sshd_config (restrict)**	`AllowGroups ssh-users`

Always restart both: `sudo systemctl restart ssh && sudo systemctl restart sshd`

Group ssh-users must exist in LDAP before AllowGroups takes effect

PART 4

Enabling POSIX Attributes in LLDAP

User Schema — add fields

Attribute	Type
uidNumber	Integer
gidNumber	Integer
homeDirectory	String
loginShell	String

Group Schema — add field

Attribute	Type
gidNumber	Integer

Set on user alex:

```
uidNumber: 10001
gidNumber: 10001
homeDirectory: /home/alex
loginShell: /bin/bash
```

Critical rule: gidNumber on the group (e.g. devops) must exactly match the gidNumber on the user, or SSSD won't resolve group membership correctly.

PART 4 — VERIFICATION

Confirming POSIX Attributes Resolve

Reset the SSSD cache

```
sudo systemctl restart sssd
sudo rm -rf /var/lib/sss/db/*
sudo systemctl restart sssd
```

Query LDAP directly

```
ldapsearch -H ldap://idm.devsecops.com:3890 \
  -D "uid=admin,..." -x -w "***" \
  -b "ou=people,dc=devsecops,dc=com" \
  "(uid=alex)"
```

Expect these tools to confirm the same picture:

`sssctl config-check` — validates sssd.conf syntax

`sssctl user-checks alex` — confirms resolution end-to-end

`getent passwd alex` — shows the resolved POSIX record

`su - alex` — logs in as the LDAP-backed user

Expected: alex:x:10001:10001::/home/alex:/bin/bash

PART 5

SSH Public Key Authentication via LDAP

No passwords over SSH | **Centralized key management** | **Instant access revocation**

1	**Extend LLDAP schema**	`User schema → add sshPublicKey (String, multi-value: YES)`
2	**Paste the public key**	`cat ~/.ssh/id_ed25519.pub → paste into user alex's profile`
3	**Map it in SSSD**	`ldap_user_ssh_public_key = sshPublicKey`

4	**Configure sshd_config**	`PubkeyAuthentication yes`
		`AuthorizedKeysCommand /usr/bin/sss_ssh_authorizedkeys`
		`AuthorizedKeysCommandUser nobody`

`Test: ssh alex@10.0.2.14 → should log in without a password prompt`

PART 6

Sudo via LDAP Groups

Reality check: LLDAP does NOT support the sudo LDAP schema — so sudo authorization is handled the plain Linux way, via group name.

Map Group → Sudo

```
sudo nano /etc/sudoers.d/devops
```

```
%devops ALL=(ALL:ALL) ALL
```

```
Test:
su - alex
sudo whoami   →  root
```

Restrict Sudo Scope

Limit which commands the group can run instead of granting full root:

```
%devops ALL=(ALL) NOPASSWD: \
  /usr/bin/apt, /usr/bin/systemctl
```

Principle of least privilege: grant only the commands a role actually needs.

PART 7

Enabling LDAPS (TLS) for LLDAP

Plain ldap://10.0.2.11:3890 sends credentials in plaintext on the wire. Not acceptable outside a throwaway lab.

1	**Generate a cert**	`openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout lldap.key -out lldap.crt -subj "/CN=idm.devsecops.com"`
2	**Update compose**	`LLDAP_LDAPS_OPTIONS__ENABLED=true, PORT=6360, CERT_FILE / KEY_FILE, then docker-compose up -d`
3	**Test LDAPS**	`ldapsearch -x -H ldaps://10.0.2.11:6360 -D "uid=admin,..." -w "***" -b "dc=devsecops,dc=com"`
4	**Update SSSD client**	`ldap_uri = ldaps://10.0.2.11:6360` `ldap_tls_reqcert = never  (lab only)`

Production note: replace the self-signed cert with a real CA, and set ldap_tls_reqcert = demand.

CHAPTER SUMMARY

What We Built & Why It Matters

What We Built	Why It Matters
LLDAP directory (users + groups)	Single source of truth for identity across the lab
SSSD + PAM + NSS integration	LDAP users become real, login-capable Linux accounts
POSIX attributes (uid/gid/home/shell)	Consistent, centrally-managed account details
SSH public key auth via LDAP	No passwords over SSH; revoke access in one place
Sudo mapped to LDAP groups	Least-privilege admin access, centrally controlled
LDAPS (TLS) encryption	Credentials no longer travel the network in plaintext

Next step: Authentik sits on top of this directory as the OIDC provider, so every downstream service — GitLab, Vault, Kubernetes — authenticates against one identity source.